

UNIVERSIDAD POLITÉCNICA METROPOLITANA DE HIDALGO

Ingeniería en Tecnologías de la Información

Tecnologías y Aplicaciones en Internet

8° B

Integrantes:

Hernandez Espinosa Zaid Emanuel
Herrera Cruz Vielka Ailyn
Lopez Lara Randy Noe
Gonzalez Lopez Jorge Saul

Profesor:

Dr. Carlos Alberto Carrillo Santos.

Tolcayuca, Hidalgo Ex Hacienda San Javier

13 ago 2026

Evidencias de Despliegue — NexaFerretería

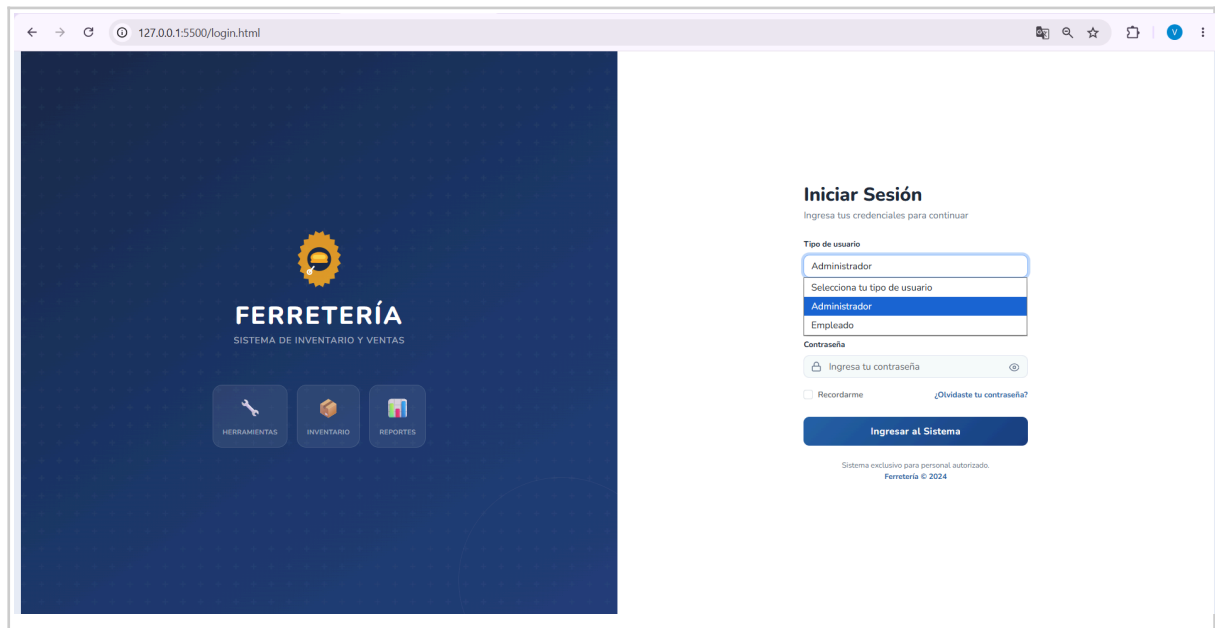
Este documento presenta la evidencia del proceso de despliegue del sistema NexaFerretería, una aplicación web de gestión de inventario para ferretería. El frontend (HTML/CSS/JS) se publicó en Vercel y el backend (Django + PostgreSQL) se publicó en Render. A continuación se muestra cada etapa con una breve explicación de lo realizado.

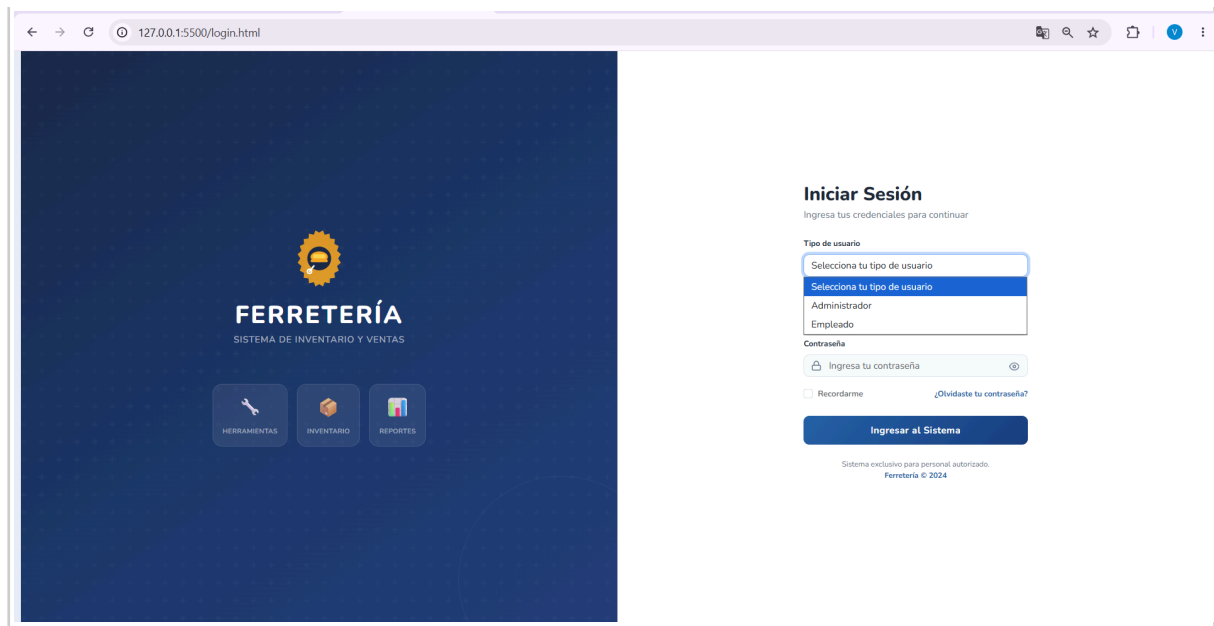
1. Despliegue del Frontend en Vercel

El frontend estático del sistema se conectó al repositorio de GitHub y se desplegó de forma automática en Vercel. Antes de subirlo, se realizaron pruebas locales de la interfaz de inicio de sesión para validar el flujo de autenticación.

1.1 Pruebas locales de inicio de sesión

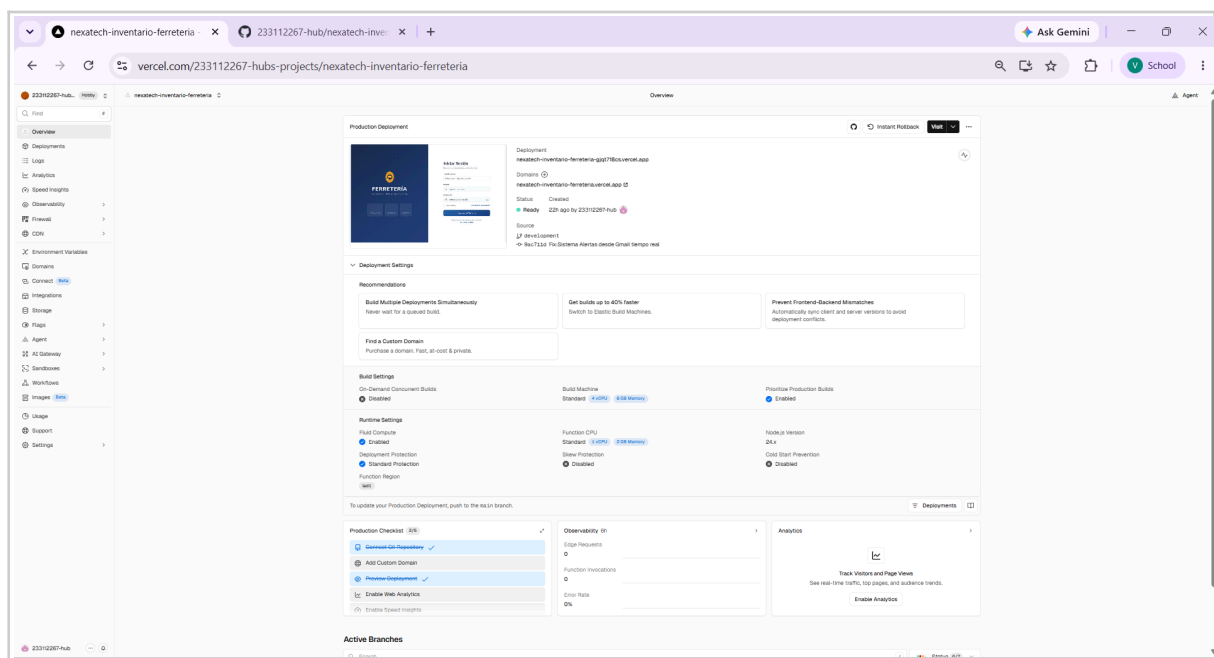
Se probó la pantalla de login de forma local (servidor en 127.0.0.1:5500) seleccionando el tipo de usuario antes de conectarla al backend en producción.





1.2 Publicación en Vercel

Una vez validada la interfaz, el proyecto se conectó a Vercel para su despliegue automático desde el repositorio de GitHub. El panel de Vercel confirma que el despliegue de producción quedó activo ("Ready") y disponible en el dominio `nexatech-inventario-ferreteria.vercel.app`.



1.3 Primera verificación contra el backend

Al abrir el frontend ya publicado en Vercel, el login mostró un error de conexión porque el backend en Render todavía no estaba desplegado ni configurado. Esta captura documenta ese punto intermedio del proceso, previo a la configuración de variables de entorno del backend.

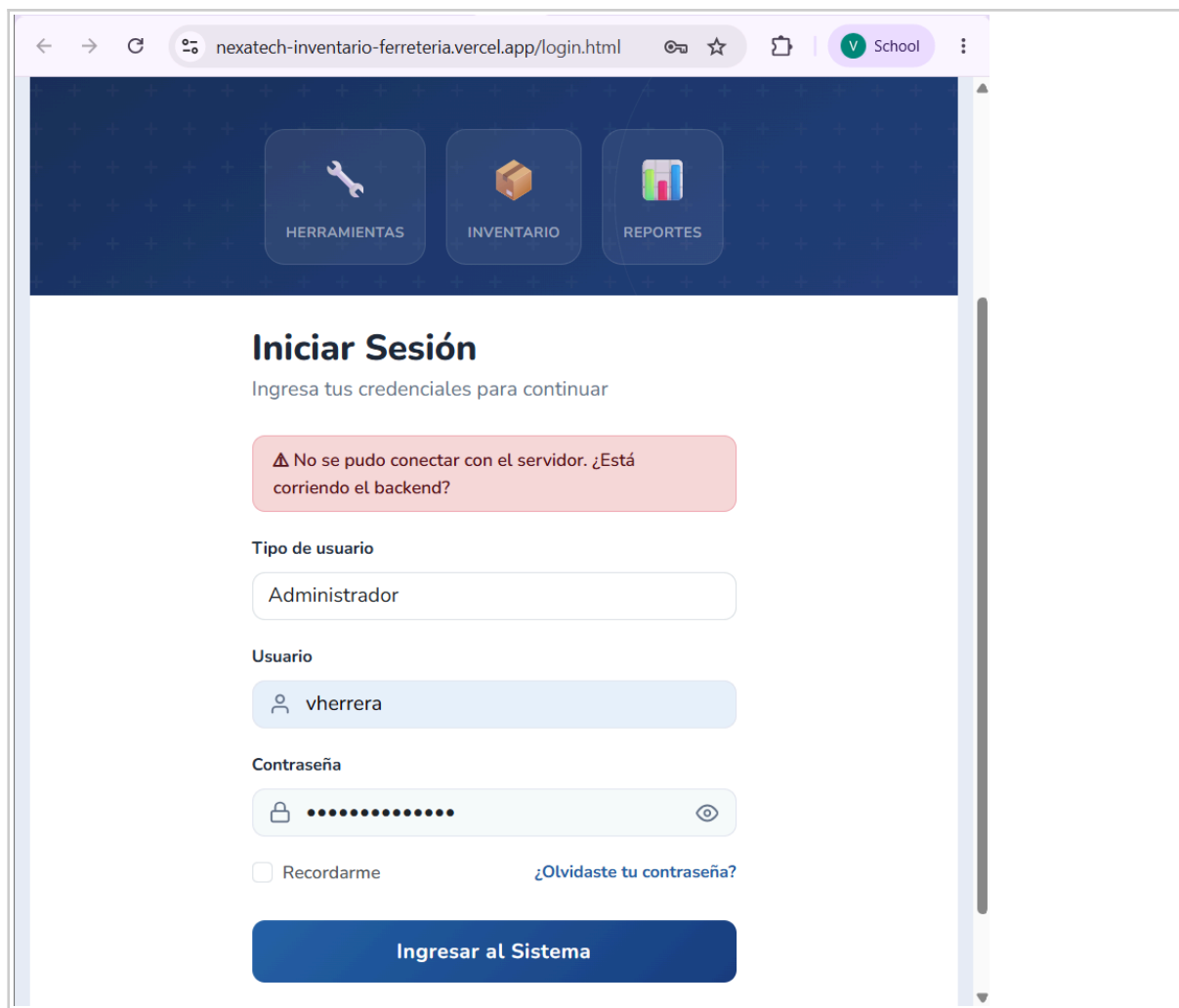


Figura 4. Error "No se pudo conectar con el servidor" al intentar iniciar sesión desde el frontend en Vercel, antes de tener el backend activo.

2. Despliegue del Backend en Render

El backend (Django REST + PostgreSQL en Neon) se desplegó como Web Service en Render. Fue necesario configurar las variables de entorno (credenciales de base de datos, llaves secretas, credenciales de correo y la URL del frontend) para que el servicio funcionara correctamente y aceptara peticiones del frontend.

2.1 Configuración de variables de entorno

Render permite cargar varias variables de entorno a la vez pegando el contenido de un archivo .env, lo que agiliza la configuración inicial del servicio.

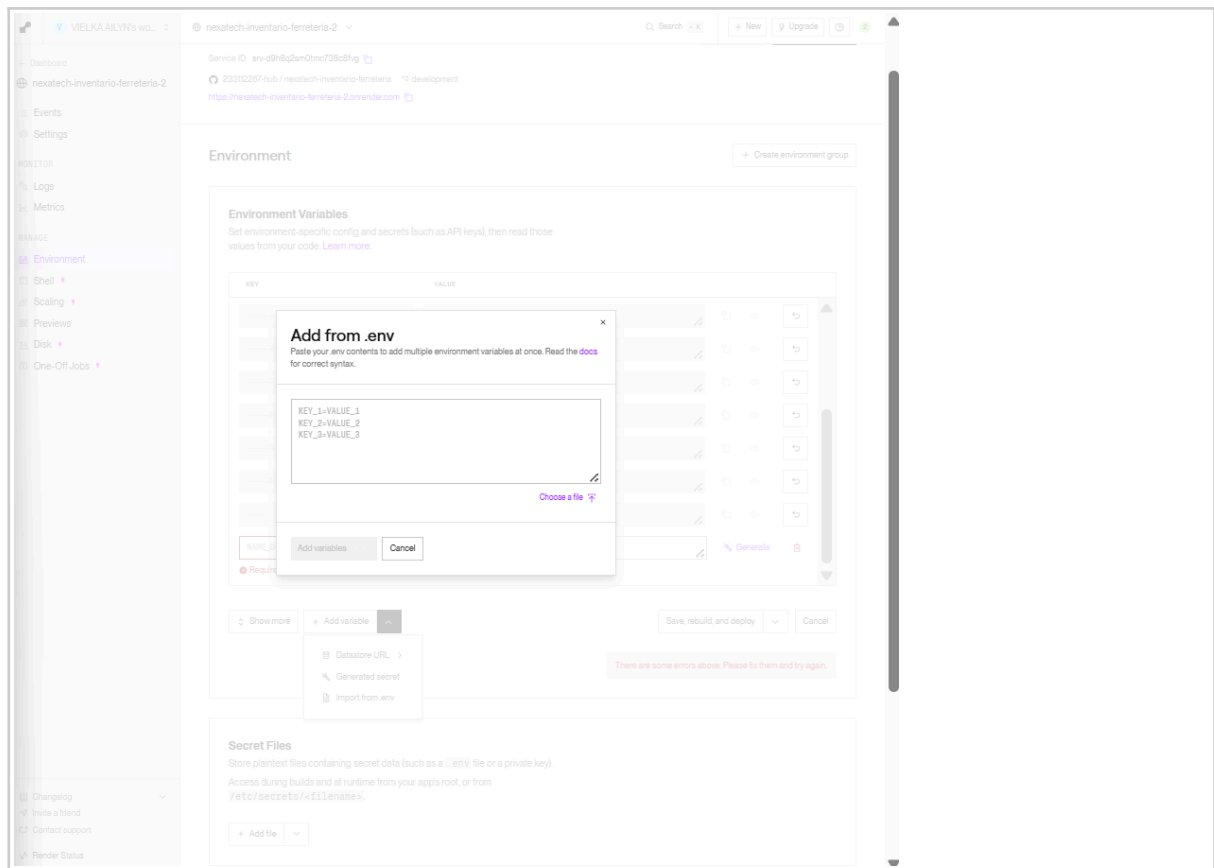


Figura 5. Cuadro de diálogo 'Add from .env' de Render para importar las variables de entorno del backend en un solo paso.

Tras la carga inicial, se revisaron las variables ya registradas (host, nombre, usuario y contraseña de la base de datos, modo DEBUG, credenciales del correo, entre otras).

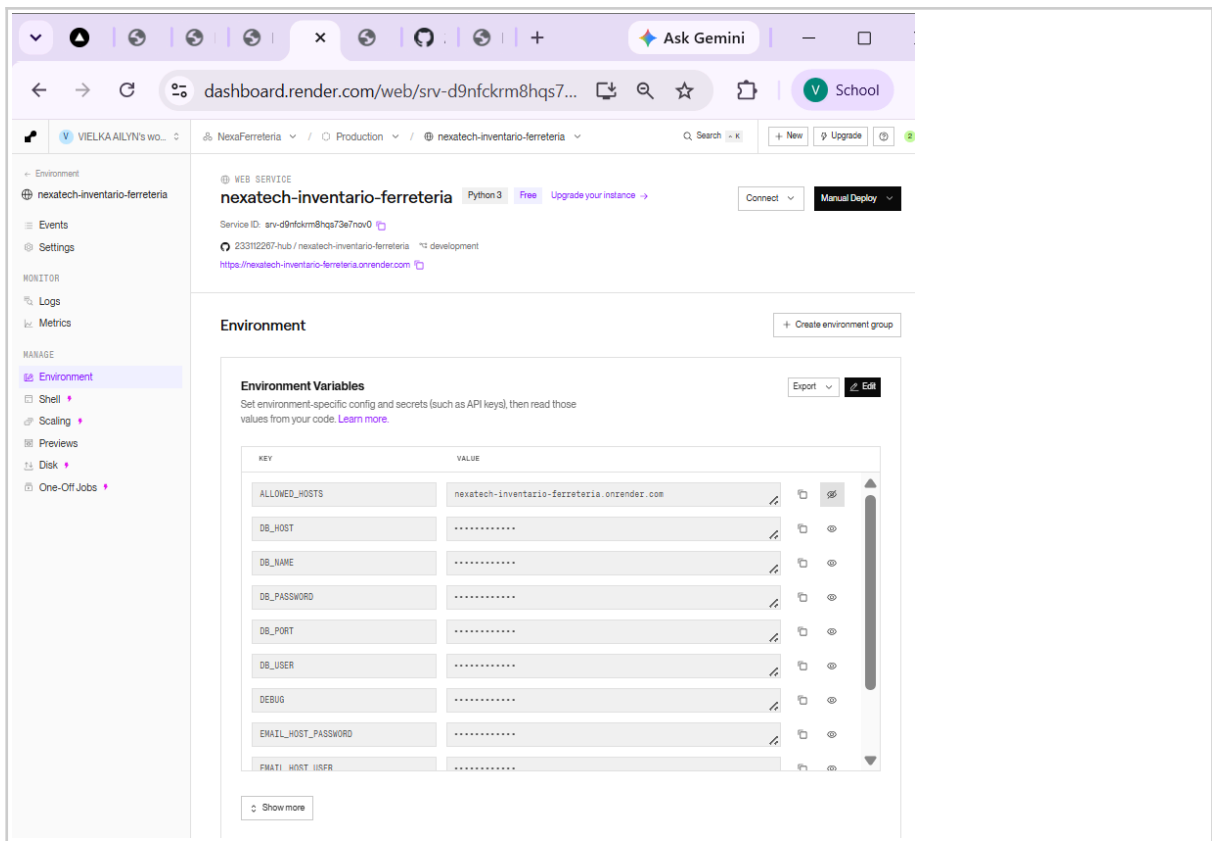


Figura 6. Lista parcial de variables de entorno cargadas en Render para el servicio backend.

Posteriormente se completó la configuración agregando la variable `FRONTEND_URL`, necesaria para habilitar CORS y permitir que el frontend en Vercel se comunice con el backend, junto con el resto de las llaves (`JWT_SECRET`, `SECRET_KEY`, credenciales de correo, etc.).

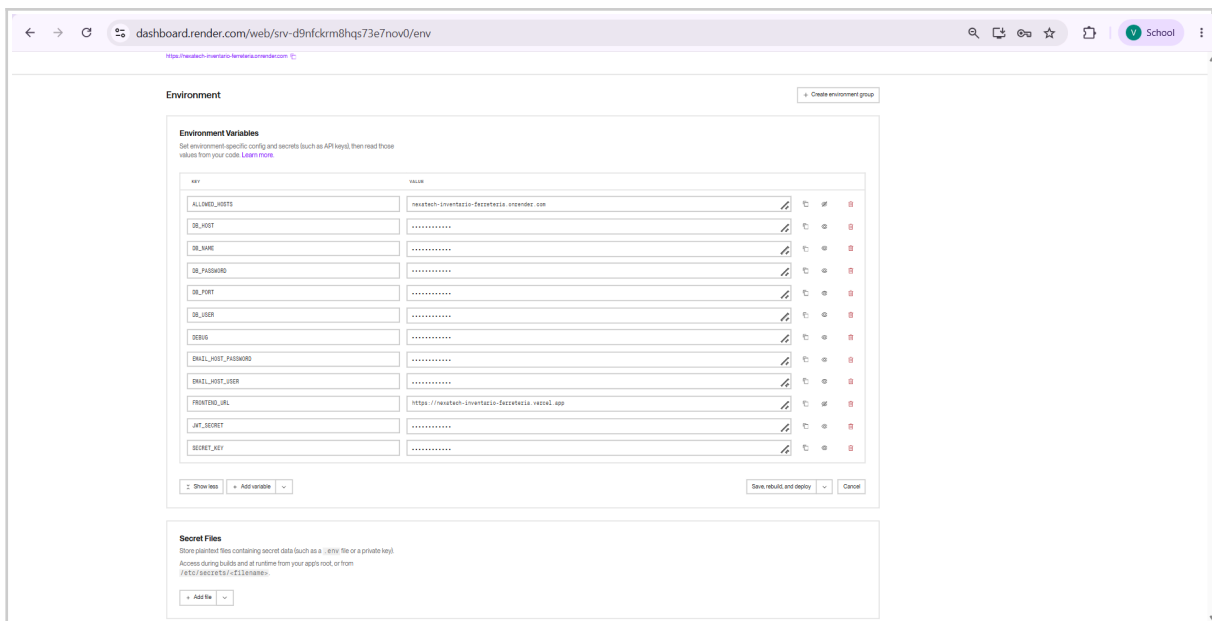


Figura 7. Conjunto completo de variables de entorno en Render, incluyendo `FRONTEND_URL` para habilitar la comunicación con el frontend.

2.2 Despliegue exitoso

Con las variables de entorno configuradas, Render reconstruyó e implementó el servicio. El registro de despliegue (logs) confirma la instalación de dependencias, la compilación exitosa ("Build successful") y el arranque del servidor con Gunicorn, quedando el servicio disponible en su URL pública.

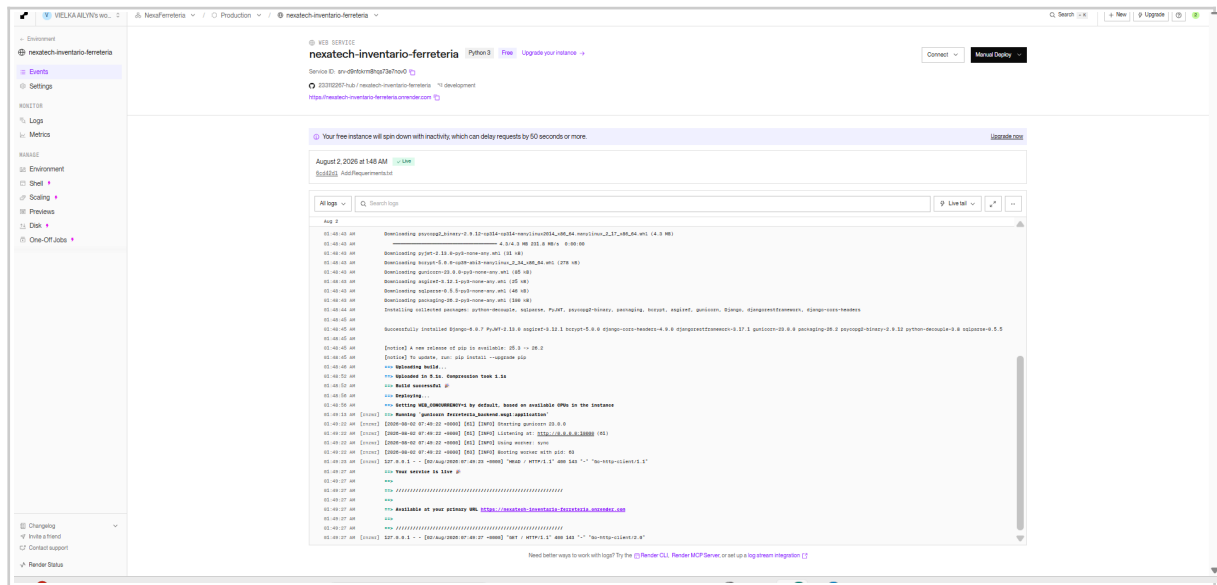


Figura 8. Logs de Render mostrando la instalación de dependencias, el build exitoso y el servicio backend ya activo ("Your service is live").

3. Verificación funcional: notificaciones por correo

Como prueba final de integración entre frontend, backend y el sistema de alertas de stock, se verificó que las notificaciones automáticas por correo electrónico (alertas de stock bajo) llegaran correctamente a la bandeja de entrada, confirmando que el envío de correos vía la API de Brevo funciona en producción.

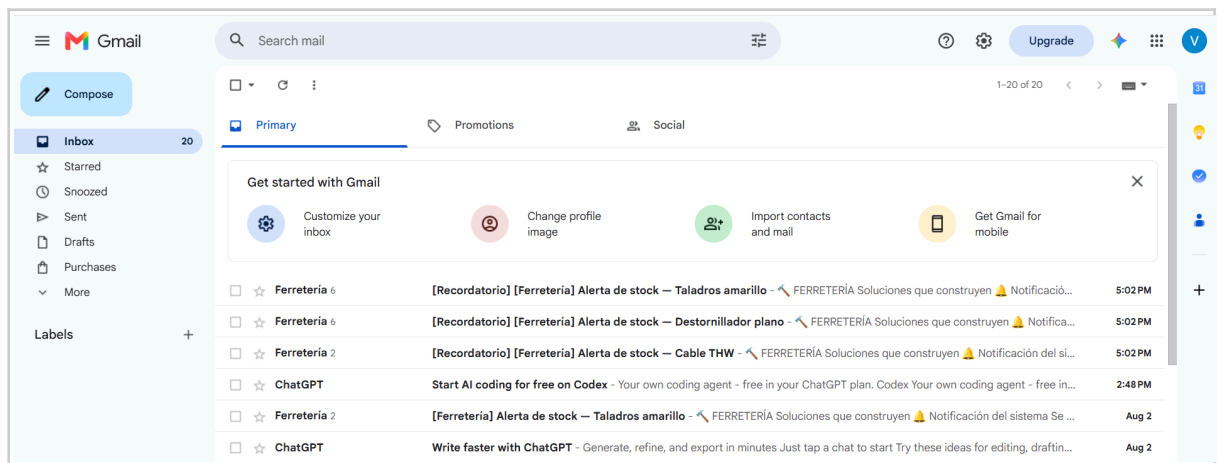


Figura 9. Bandeja de entrada de Gmail recibiendo los correos de alerta de stock generados automáticamente por el sistema en producción.

4. Conclusión

Con estas evidencias se confirma que el sistema NexaFerretería quedó correctamente desplegado en un entorno de producción distribuido: frontend en Vercel y backend en Render con base de datos PostgreSQL, incluyendo la comunicación entre ambos servicios (CORS) y el funcionamiento del sistema de notificaciones por correo electrónico.